

Software Architectures

Assignment 2: Actor-based Design Patterns

TA: Camilo Velázquez-Rodríguez
camilo.ernesto.velazquez.rodriguez@vub.be
F.10.725

Assignment

For this assignment you will implement and report on your original solution to an architectural problem.

Deadline: January 5th 2025 by 23:59. The deadline is fixed and will not be extended for any reason. Submissions after the deadline will not be considered for grading and the grade will automatically be **ABS**.

Deliverables A report and source code. The report (in English) explains your solution to the problem and the main components used in your implementation. Please use simple diagrams to illustrate the architecture of your solution.

The report should be handed in as a single PDF file.

The file should follow the naming schema `FirstName-LastName-SA2.pdf`, for example: `Camilo-Velazquez-SA2.pdf`.

The source code is your project implementation zipped in a single file. Do not include files or folders resulting from the compilation process of your project, e.g., `target/` or `*.class`. Submit the *report* and *source code* as a single zip file to the Software Architectures course page on Canvas, by clicking on [Assignments > Assignment 2](#).

Plagiarism Any suspicion of plagiarism will be reported to the Dean of Faculty without delay. Both user and provider of such code will be reported and will be dealt with according to the plagiarism rules of the examination regulations (cf., OER, Article 118). The Dean may decide on (a combination of) the disciplinary sanctions, ranging from 0/20 on the paper of the given programme unit to a prohibition from (re-)enrolling for one or multiple academic years (cf., OER, Article 118\$5).

Grading Your solution will be graded and can become the subject of an additional defence upon the lecturer's request.

Problem Description

For this assignment, you will design and implement an online auction system (e.g., similar to eBay) based on a microservice architecture. In the auction system, bidders can make and cancel bids whereas sellers can create and remove auctions.

The setup of your implementation should comprise at least the following set of actors:

- A **Seller** capable of creating and removing auctions.
- An **Auction** with the item to be auctioned, its price, the list of all bids sorted in decreasing order, and a time indicating the availability of the auction.
- A **Bidder** with a name and a bank account number. A **Bidder** can make and withdraw bids on any of the available auctions.
- **eBay**, which maintains a collection of currently available auctions and their highest bids.
- A **Bank** having a list of registered accounts.

A more detailed description of each actor, their messages and interactions follows:

A **Seller** can create **Auctions** after which they become available on **eBay**. The **Seller** could also cancel **Auctions** if the number of bids is insufficient for them (e.g., only one bid or a bid that is too low). Each **Auction** should have the item to be bid on and its initial price. It should also maintain a list of past bids for the item. More importantly, each **Auction** is available for a predetermined time.

The availability time allows bidders to submit their bids for the auction for the duration stated by the **Seller** when the **Auction** is created. After the auction closes, no further bids should be accepted. Lastly, an **Auction** should notify each of its **Bidders** every time a bid surpasses the current top bid.

The **eBay** system will maintain the list of auctions available at any point in time. For example, if **eBay** is requested to provide the list of currently running auctions, it should respond with the available auctions, their items and their highest bid until that moment. In case a new auction is created, it should update the list and, conversely, if the winner has paid for the auction, the list should also be updated accordingly.

The **Bidder** should request **eBay** for the list of items being auctioned. After obtaining the list, it should select one random item and make a bid on it by sending the amount it is willing to pay to the **Auction** and its credentials (e.g., name, bank number, etc.). The **Bidder** might also cancel a previously made bid. To cancel, the **Bidder** should send a cancellation message to the **Auction**, and it should be accordingly removed from the list of bids.

The **Bank** should maintain the list of registered bank accounts. When the time for an auction is completed, the **Auction** will contact the **Bank** with the bank account number and the name of the highest **Bidder**. If the credentials do not match or the **Bank** does not pay in time, the **Auction** will have to be re-bid again as described before. In case everything goes well, the **Bank** should notify both the **Seller** and the **Bidder** about the

successful purchase. The auctioned item will not be officially sold until the **Bank** receives acknowledgement of the **Seller** and the **Bidder** having been notified.

Finally, after the item acquisition, the **Bidder** might have second thoughts about it, since, for example, the item might have not fulfilled the expectations. In this case, the **Bidder** should have a grace period to return the acquired item. After the return period has passed, sending the item back is no longer possible. The item should be returned to the **Seller** who initially sold it. Consequently, the **Bank** should put the spent money, back into the **Bidder**'s account. After receiving the item, the **Seller** should consider to re-auction it or not. The bidders who did not win the previous auction on the returned item should be contacted for a new bid.

All actors and their states should be persisted locally in case the **eBay** system goes down. This is to not lose any ongoing auctions and their respective bids. Demonstrate in your solution a sudden interruption of the system and its re-deployment using the techniques studied in the course.

Make sure your solution supports at least 5 **Sellers**, 10 **Auctions** and 10 **Bidders**. In addition to the general approach, you will be evaluated according to how you implemented your actor-based architecture. **Motivate your design decisions** in the report as per the problem description. For example, justify the selection of one or more architectural patterns (e.g., Forward Flow, Aggregator, Event-Sourcing) in the report. Illustrate your approach using concepts from Akka actors and actor-based design patterns.

More information on Actors and Akka

- <https://doc.akka.io/docs/akka/current/typed/>
- <https://doc.akka.io/docs/akka/current/typed/actors.html>